

Redireccionamiento y Pipes

Redireccionamiento

Con el manejo de *streams* en Linux se comprende de forma más precisa el uso del **redireccionamiento**. Pudiendo manejar los flujos de entrada o de salida de los comandos o programas, Linux nos permite almacenar el resultado de ellos en un archivo para luego analizarlo o bien enviar los errores en otro archivo que necesitemos ver luego. Incluso podríamos enviar la salida de un comando como entrada de otro comando para finalmente obtener un resultado eficaz.

Existen cuatro tipos de redireccionamiento:

- 1) >: Mayor
- 2) >>: Doble mayor
- 3) <: Menor
- 4) |: Pipe

>: Mayor

El > (mayor) permite que en vez de que el resultado de un comando se muestre en pantalla, se pueda almacenar en un archivo u otro dispositivo.

Ejemplo

```
ls -l /etc > archivo_ls
```

Si el archivo no existiese, este se crea automáticamente y rellena el contenido con lo que le enviamos. Si, por el contrario, el archivo ya estuviera creado, este sería eliminado y vuelto a generar con lo que le enviemos.

>>: Doble Mayor

El >> (Doble Mayor) permite **añadir** el contenido de la salida a un archivo ya existente o crearlo en caso de no existir.

Ejemplo

```
ls -l /var >> archivo_ls
```

En este caso, lo que sucederá es que al final de nuestro archivo llamado “archivo_ls” donde direccionamos la salida del contenido del directorio /etc, podremos **añadir** al final de la última línea, la salida del comando “ls -l”, que muestra el contenido del directorio /var a continuación.

<: Menor

El < (Menor) permite al comando leer la información a procesar desde un archivo u otro dispositivo en lugar de leer los datos directamente desde el teclado o el stdin.

Ejemplo

```
cat < archivo_ls
```

Esto indica que el comando “cat” debe leer desde “archivo_ls” en lugar del teclado.

Redireccionamiento de stderr (Standard Error)

Determinando que el stream correspondiente al StdErr está identificado con el número 2, podemos utilizarlo para poder redirigir los errores hacia un archivo.

Para esto agregaremos el número del stderr a la redirección del mayor:

Ejemplo

Si nosotros queremos listar un directorio inexistente, obtendremos el siguiente mensaje de error:

```
ls /directorio_nuevo
```

```
ls: cannot access '/directorio_nuevo': No such file or directory
```

Pero si redirigimos el stderr de la siguiente forma, no obtendremos por el stdout el mensaje, sino que lo enviaremos al archivo que se acaba de crear llamado “error.log”:

```
ls /directorio_nuevo 2> error.log
```

Que podremos visualizar de la siguiente manera y comprobar su contenido:

```
cat error.log
```

```
ls: cannot access '/directorio_nuevo': No such file or directory
```

Vale decir que cuando usamos el símbolo > para redirigir el stdout a un archivo, implícitamente estamos asumiendo la colocación del número 1 delante del mayor de la siguiente forma:

```
ls /directorio_nuevo 1> salida.log
```

Otros tipos de redireccionamiento

Dentro de Linux hay un dispositivo conocido como “**null**” que se encuentra en el directorio /dev. Este dispositivo actúa como “agujero negro” dentro del sistema operativo. Esto quiere decir que cualquier cosa enviada ahí, es pérdida para siempre, convirtiéndose de utilidad para cuando queremos descartar los errores y sólo quedarnos con los mensajes que nos son de utilidad.

A continuación, mostraremos un tipo de redireccionamiento múltiple:

stdout es enviado a salida.txt y stderr a error.log:

```
ls /directorio_nuevo >> salida.txt 2> error.log
```

stdout es enviado a salida.txt y stderr es descartado:

```
ls /directorio_nuevo >> salida.txt 2> /dev/null
```

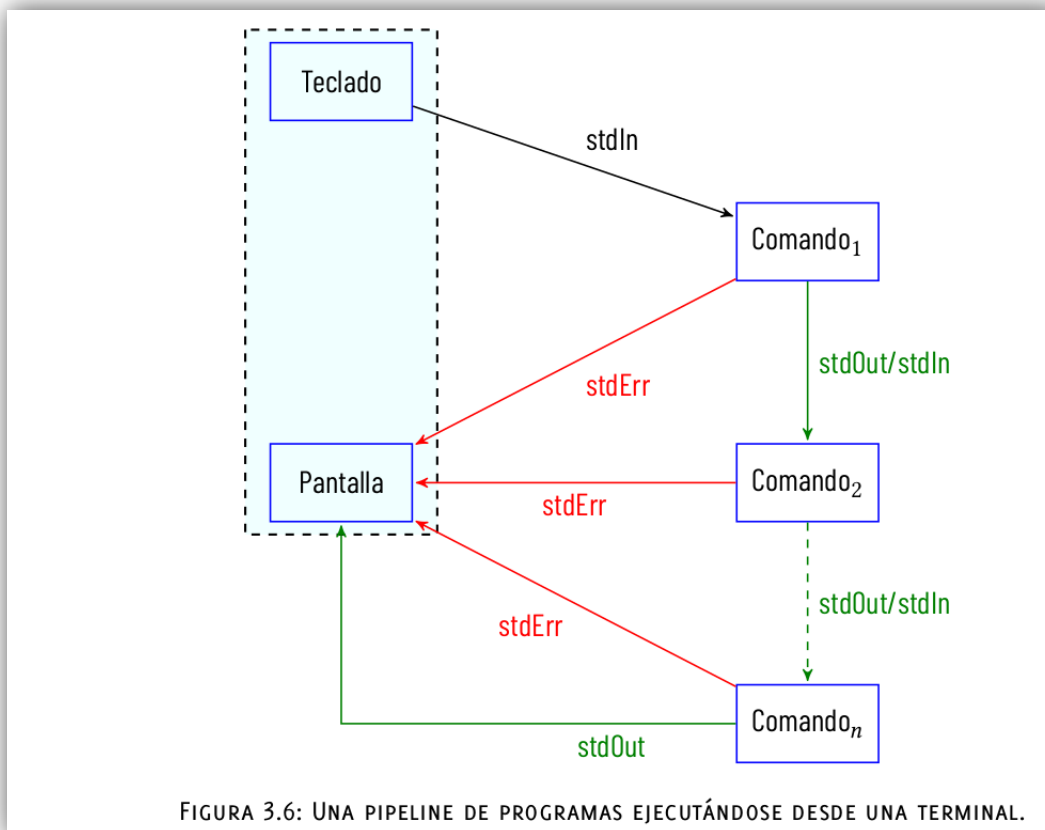
stdout y stderr es enviado a salida.txt:

```
ls /directorio_nuevo &> salida.txt (en versiones de bash 4.0 en adelante).
```

```
ls /directorio_nuevo > salida.txt 2>&1 (compatibilidad con versiones anteriores de bash y otros shells).
```

Pipes

El Pipe | permite enviar la salida de un comando como entrada de otro, encadenando comandos como sea necesario. De esta manera, la salida de cada comando (stdout) se convierte en la entrada del siguiente (stdin).



Fuente: Niklas, P. (2020). *GNU/Linux, Con sabor a Debian*. Pág. 82 (3.2.6).

Cuando se analizan logs o cualquier tipo de archivo que contenga múltiples líneas, puede llegar a ser muy útil la combinación del uso de *pipes* con filtros, obteniendo información de los archivos de forma más eficiente. Algunos ejemplos de cómo podrían usarse:

`cat /var/log/dpkg.log | grep "installed"`: buscará en el archivo dpkg.log toda línea que cumpla con el patron "installed".

`cat archivo_ls | head -n 2`: mostrará las 2 primeras líneas del archivo llamado "archivo_ls".

Buscar en Linux

Para realizar búsquedas dentro de la estructura de archivos y directorios en Linux, podemos utilizar buscadores como ***find*** o ***locate***.

find

Permite buscar archivos y directorios dentro del sistema de archivos de manera exhaustiva. Es muy versátil y flexible, ya que permite buscar en función de varios criterios como el nombre, la fecha de modificación, el tipo de archivos, el tamaño, entre otras tantas opciones. Además de estas opciones, **find** puede ejecutar acciones específicas teniendo como parámetros, los archivos encontrados.

Ejemplo

```
find [ruta] [opciones] [acción]
```

- **ruta:** directorio donde comienza la búsqueda, si no se especifica, comienza en el lugar donde estemos posicionados.
- **opciones:** opciones que se le pueden pasar para controlar la búsqueda:
 - **-name:** busca archivos por nombre específicamente.
 - **-iname:** busca archivos por nombre, ignorando mayúsculas y minúsculas.
 - **-type f:** buscar archivos (**files**).
 - **-type d:** busca directorios (*directory*).
 - **-size:** busca por tamaño:
 - **-size +100M:** ejemplo archivo mayores a 100 MB.
 - **-size -500k:** ejemplo archivos menores a 500 KB.
 - **-mtime:** busca archivos modificados hace cierto número de días:
 - **-mtime -7:** ejemplo archivos modificados en los últimos 7 días.
 - **-mtime +30:** ejemplo archivos modificados hace más de 30 días.
 - **-perm:** busca archivos con permisos específicos:
 - **-perm 755:** ejemplo archivo con permisos 755.

- **Acción:** qué hacer con los archivos encontrados, podría ser eliminarlos, moverlos, comprimirlos, etc.
 - **-exec:** ejecuta un comando sobre cada archivo encontrado. Se debe utilizar {} como marcador de posición para el nombre del archivo.
Ejemplo: `find /home/alumno -name "*.txt" -exec cp {} /backup/ \;`
Copia los archivos que estén dentro del directorio especificado, con la extensión txt al directorio backup.
 - **-delete:** elimina los archivos encontrados.
Ejemplo: `find /var/log -name "*.log" -delete`
Borra del directorio especificado los archivos .log

locate

El comando **locate** es parte de un programa llamado "mlocate". Es mucho más rápido que **find** porque utiliza una base de datos propia, previamente indexada de los archivos del sistema. Esto permite obtener resultados en cuestión de segundos, pero depende de la actualización de la base de datos.

Nota: mlocate es un programa que generalmente no está instalado por defecto. Para usarlo, previamente hay que instalarlo. Luego se debe actualizar la base con el comando "updatedb" (siendo *root* o teniendo permisos de administrador).

Ejemplo

```
locate [nombre del archivo]
```

Comparación: *find* y *locate*

Comando	Ventaja principal	Desventaja
<i>find</i>	Búsqueda exhaustiva, permite usar criterios avanzados y ejecutar acciones.	Más lento, ya que busca en tiempo real en el sistema de archivos.
<i>locate</i>	Muy rápido, útil para búsquedas generales.	Depende de una base de datos que debe actualizarse para estar al día.

Cuadro de elaboración propia.

Bibliografía

- Niklas, P. (2020) *GNU/Linux, con sabor a Debian* - Primera Edición —Pág. 63 (2.5) a Pág. 86.